



Accretion

SECURITY ASSESSMENT



Sanctum

Provided by Accretion Labs Pte Ltd. for Sanctum

February 25, 2026

A25SAN2

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Niklas Brymko (niklas@accretion.xyz)
Auditor	David Ratiney (david@accretion.xyz)

CLIENT

Sanctum (<https://sanctum.so>) engaged Accretion to conduct a comprehensive security audit of sanctum's inf-v2 protocol smart contracts and associated infrastructure components.

ENGAGEMENT TIMELINE

15 Dec	Project Kickoff Initial planning and scope definition
15 Dec	Assessment Begins Security review and testing phase
15 Jan	Review Fixes Security recommendations are given and implemented
25 Feb	Project Completion Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	5ocnV1qiCgaQR8Jb8xWnVbApfaygJ8tNoZfgPwsgx9kx	https://github.com/igneous-labs/inf-1.5

ASSESSMENT

The security assessment of sanctum's inf-v2 revealed 6 total findings with no critical or high severity issues. The breakdown includes 3 low severity and 3 informational issues. Only 1 informational issue was fixed, while 4 low/info issues remain as wontfix, indicating acceptable risk levels.

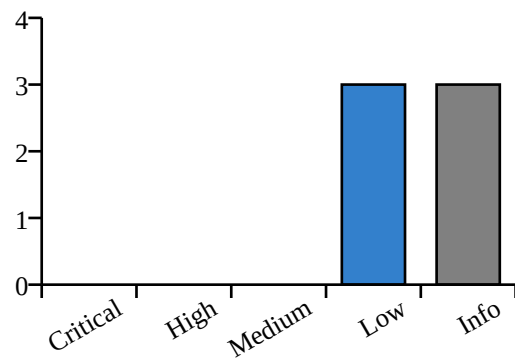
CODE ASSESSMENT

The codebase demonstrates solid security practices with proper SPL token handling and PDA management. However, minor Token-2022 compatibility issues and insufficient oracle validation present areas for improvement in the protocol's architecture.

KEY FINDINGS

No critical or high severity vulnerabilities were identified. All findings were categorized as low severity or informational, indicating the protocol maintains a strong security posture without immediate threats to user funds or system integrity.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Inf V2 Controller

Link: <https://github.com/igneous-labs/inf-1.5>

Commit: 501f68ea005e49481888b4d237ba69f69653aa68

Program ID: 5ocnV1qiCgaQR8Jb8xWnVbApfaygJ8tNoZfgPwsgx9kx

Audit Result:

- **Audited Commit:** d2019745d4ef26d7d4d97f894bcfdc39d2428ef6

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-L1	Token-2022 Transfer Fee Extension Tokens Will Fail	LOW	WONTFIX
ACC-L2	Value calculator programs are not validated further than being executable.	LOW	WONTFIX
ACC-L3	INF LP Token Supply Inflation via Circular AddLiquidity	LOW	WONTFIX
ACC-I1	Token-2022 Transfer Hook Tokens Not Explicitly Handled	INFO	WONTFIX
ACC-I2	Rebalance Record are not Rent exempt and rely on Undefined Behaviour to be kept alive	INFO	UNKNOWN
ACC-I3	Privileged Authority Rent extraction	INFO	FIXED

DETAILED ISSUES

ACC-L1 Token-2022 Transfer Fee Extension Tokens Will Fail

LOW

WONTFIX

Description

We found that the protocol allows Token-2022 mints but uses `transfer_checked` instead of `transfer_checked_with_fee`. For tokens with the Transfer Fee extension enabled, the actual amount deposited to pool reserves will be less than the specified amount. The protocol calculates output amounts based on the input amount, not the actual received amount.

While this won't cause fund loss (the `PoolWouldLoseSolValue` check in `final_sync` will catch the discrepancy and revert the transaction), it means Token-2022 tokens with transfer fees are effectively incompatible with this protocol.

Location

<https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/swap/v2/common.rs#L318-L353>

Relevant Code

```
SwapV2Ctl::AddLiq(accs) | SwapV2Ctl::Swap(accs) => cpi.invoke_fwd_handle(
    abr,
    *accs.ix_prefix.inp_token_program(),
    TransferCheckedIxData::new(
        *inp,
        checked_mint_of(abr.get(*accs.ix_prefix.inp_mint()))?.decimals(),
    )
    .as_buf(),
    // ...
),
```

Mitigation Suggestion

Either: 1. Document that Token-2022 tokens with Transfer Fee extension are not supported 2. Or implement `transfer_checked_with_fee` handling with proper fee accounting

Remediation

TODO: remediation with link to commit

Description

The `sol_value_calculator` is a critical price oracle that determines:

- SOL value of LST reserves (LstToSol)
- How much INF to mint/burn for liquidity operations
- Yield calculations (`withheld_lamports`)
- Total pool value (`total_sol_value`)

Only `is_executable()` is used as the current validation and any deployed program can be set. This creates issues when foreign-controlled programs are used, even if they appear to be non-malicious. If a `sol_value_calculator` program is upgraded maliciously (by a compromised/malicious upgrade authority):

| Attack | Method | Impact | |-----|-----|-----| | Inflate value |
Return inflated SOL value | Attacker removes more LST than deserved | | Deflate value | Return deflated SOL value |
Attacker swaps in cheap, gets expensive LST | | Drain pool | Manipulate during rebalance | Bypass `new_total >= old_total` check | | Steal yield | Inflate then deflate | Siphon `withheld_lamports` |

Location

https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/admin/set_sol_value_calculator.rs#L115-L115

Relevant Code

```
#[inline]
fn verify_is_program(
    should_be_program: &Account,
    faulty_err: Inf1CtlErr,
) -> Result<(), ProgramError> {
    match should_be_program.is_executable() {
        true => Ok(()),
        false => Err(Inf1CtlCustomProgErr(faulty_err).into()),
    }
}

#[inline]
pub fn verify_sol_value_calculator_is_program(calc_program: &Account) -> Result<(), ProgramError> {
    verify_is_program(calc_program, Inf1CtlErr::FaultySolValueCalculator)
}
```

Mitigation Suggestion

If this should only ever be the pricing program, compare the `program_id`.

Remediation

TODO: remediation with link to commit

Description

The `add_lst` instruction allows the admin to add any valid SPL token mint to the `LstStateList`, including the INF LP token itself (`pool.lp_token_mint`). There is no validation preventing this circular configuration.

If INF is added to the `LstStateList` with a `sol_value_calculator` and 0% fees configured in the pricing program, an attacker can exploit this to inflate the INF supply:

1. Call `AddLiquidity` with INF as input 2. X INF tokens transfer to pool reserves 3. X INF tokens minted back to attacker (with 0% fees) 4. Net attacker balance: unchanged 5. Net INF supply: increased by X

The attacker can loop this indefinitely since they never lose tokens. Each iteration increases supply by the deposit amount. With a large INF position and enough transactions, an attacker could approach `u64::MAX` supply, at which point:

- New INF mints would overflow
- No users could add liquidity
- Protocol effectively DoS'd

Location

https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/admin/add_lst.rs#L135-L143

Relevant Code

```
/// add_lst.rs L135-L143
new_lst_state = LstState {
    is_input_disabled: 0,
    pool_reserves_bump,
    protocol_fee_accumulator_bump,
    padding: [0u8; 5],
    sol_value: 0,
    mint, // No check that mint != pool.lp_token_mint
    sol_value_calculator,
};
```

Mitigation Suggestion

```
// In process_add_lst, after pool_state_v2_checked: let pool = pool_state_v2_checked(abr.get(*accs.pool_state()))?;
```

```
// Add this check: if *abr.get(*accs.lst_mint()).key() == pool.lp_token_mint { return
Err(Inf1CtlCustomProgErr(Inf1CtlErr::CannotAddLpTokenAsLst).into()); }
```

Remediation

TODO: remediation with link to commit

Description

We found that the protocol uses `transfer_checked`, which will invoke any Transfer Hooks configured on Token-2022 tokens. While this is generally correct behavior, transfer hooks could add additional accounts requirements not anticipated by the protocol. The protocol does not explicitly check for or handle tokens with Transfer Hook extensions.

Location

<https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/swap/v2/common.rs> <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/rebalance/start.rs>

Relevant Code

```
/ Lines 338-353 - Input token transfer (AddLiq/Swap):
SwapV2Ctl::AddLiq(accs) | SwapV2Ctl::Swap(accs) => cpi.invoke_fwd_handle(
    abr,
    *accs.ix_prefix.inp_token_program(),
    TransferCheckedIxData::new(
        *inp,
        checked_mint_of(abr.get(*accs.ix_prefix.inp_mint()))?.decimals(),
    )
    .as_buf(),
    NewTransferCheckedIxAccsBuilder::start()
        .with_auth(*accs.ix_prefix.signer())
        .with_src(*accs.ix_prefix.inp_acc())
        .with_dst(*accs.ix_prefix.inp_pool_reserves())
        .with_mint(*accs.ix_prefix.inp_mint())
        .build()
        .0,
),

// Lines 371-388 - Output token transfer (RemLiq/Swap):
SwapV2Ctl::RemLiq(accs) | SwapV2Ctl::Swap(accs) => cpi.invoke_signed_handle(
    abr,
    *accs.ix_prefix.out_token_program(),
    TransferCheckedIxData::new(
        *out,
        checked_mint_of(abr.get(*accs.ix_prefix.out_mint()))?.decimals(),
    )
    .as_buf(),
    transfer_checked_ix_account_handle_perms(
        NewTransferCheckedIxAccsBuilder::start()
            .with_auth(*accs.ix_prefix.pool_state())
            .with_src(*accs.ix_prefix.out_pool_reserves())
            .with_dst(*accs.ix_prefix.out_acc())
            .with_mint(*accs.ix_prefix.out_mint())
            .build(),
    ),
    &[POOL_STATE_SIGNER],
),
```

Mitigation Suggestion

Consider: 1. Documenting that Transfer Hook tokens may have unpredictable behavior 2. Or explicitly checking for and rejecting tokens with Transfer Hook extension if not intended to be supported

Given that LST's need to be added explicitly, it's best to add this check in the **AddLst** instruction.

Remediation

TODO: remediation with link to commit

Description

We found that the static Lamport transfer of 1 is lower than the Rent needed for the **RebalanceRecord**. When an account is purged, in this case, it is not properly defined and could happen in between instructions in the rebalancing. It's best to fund the account to rent exemption to prevent issues down the line.

Location

<https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/rebalance/start.rs#L304-L305>

Relevant Code

```
/// start.rs L304-L305
// hot potato
abr.transfer_direct(*ix_prefix.pool_state(), *ix_prefix.rebalance_record(), 1)?;
```

Mitigation Suggestion

Fund the account to be rent-exempt.

Remediation

TODO: remediation with link to commit

Description

We found that only the Admin can add Pool Disable Authorities; the Admin also pays the increased rent for the PDA. However, Pool Disable Authorities can remove themselves from that list, pocketing the rent increase.

Location

<https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/utils.rs#L80-L103>

Relevant Code

```
/// utils.rs L80-L103
#[inline]
fn extend_packed_list_pda<T>(  
    abr: &mut Abr,  
    cpi: &mut Cpi,  
    accs: &TransferIxAccs<AccountHandle>,  
    rent: &Rent,  
    signer: PdaSigner,  
) -> Result<(), ProgramError> {  
    if abr.get(*accs.to()).data_len() == 0 {  
        assign_invoke_signed(  
            abr,  
            cpi,  
            NewAssignIxAccsBuilder::start()  
                .with_assign(*accs.to())  
                .build(),  
            &ID,  
            &[signer],  
        )?;  
    }  
    let list_acc = abr.get_mut(*accs.to());  
    list_acc.grow_by(size_of::<T>(), false)?;  
    pay_for_rent_exempt_shortfall(abr, cpi, accs, rent)?;  
    Ok(())  
}  
  
/// abr.rs L116-L128  
#[inline(always)]  
pub fn close<'account>(  
    &mut self,  
    close: AccountHandle<'account>,  
    refund_rent_to: AccountHandle<'account>,  
) -> Result<(), ProgramError> {  
    let balance = self.get(close).lamports();  
    self.transfer_direct(close, refund_rent_to, balance)?;  
    let close_acc = self.get_mut(close);  
    close_acc.realloc(0, false)?;  
    close_acc.assign_direct([0u8; 32]); // TODO: use const pubkey for system program  
    Ok(())  
}
```

Mitigation Suggestion

Only refund rent if the signer is the Admin.

Remediation

Remediated in <https://github.com/igneous-labs/inf-1.5/pull/162/changes>

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.